

ECE8493 Introduction to Neural Networks

Project 4: Iterative Principal Component Analysis on MNIST

Digit 0 Reconstruction

Student Name: Meshaal Mouawad

Student ID: mm4922

Date: April 18, 2026

*Submitted in partial fulfillment of the requirements for
ECE8493 Introduction to Neural Networks*

Due Date: April 13, 2026

Abstract

This project implements iterative Principal Component Analysis on the MNIST dataset. We focused only on digit zero to help reduce the workload. We implemented two learning methods, the first was one-step learning using Oja's rule, the second was two-step learning with explicit normalization. The online version of the algorithm gave us NaN errors, which we switched to a batch update version. That fixed the numerical instability.

The dot product between the two methods was 1.000000, which means they converged to identical principal components. The reconstruction error went down, with one PC it was 3416.86, and with two PCs it dropped to 2896.58. That is a reduction of 15.23%. The second PC was orthogonal to the first, with dot product was 0.000000.

Contents

Abstract	1
1 Introduction	3
1.1 Project Requirements	3
1.2 Why Only Digit Zero?	4
2 Algorithm Description	4
2.1 Data Preprocessing	4
2.2 One-Step Learning Using Oja's Rule	4
2.3 Two-Step Learning with Explicit Normalization	4
2.4 Finding the Second Principal Component	5
2.5 Reconstruction and Error Computation	5
3 Results	5
3.1 Experiment 1: First Principal Component	5
3.2 Experiment 2: Second Principal Component	6
3.3 Experiment 3: Reconstruction Examples	6
4 Discussion	8
4.1 Challenges Encountered	8
4.2 Why Two Principal Components Improve Reconstruction	8
4.3 Convergence Behavior	9
4.4 Proposed Improvements	9
5 Conclusion	9
List of Figures	11
Appendix: MATLAB Code	12

1 Introduction

MNIST stands for Modified National Institute of Standards and Technology. MNIST is a database of handwritten digits used for training and testing many image processing systems and machine learning research [6]. It has 60,000 training images and 10,000 testing images of handwritten digits [6]. Later in 2017, an extended database has been published that has 240,000 training images and 40,000 testing images. Since MNIST is a standard training dataset for digits in English, in recent times, others have also provided similar databases for training digit datasets in other languages. The dataset is considered as a benchmark for neural networks worldwide. The MNIST database is good for people who want to try machine learning techniques and neural network methods on real-world data while spending minimal efforts on preprocessing and formatting [6]. It reduces the time and effort spent on preprocessing and formatting of data. The MNIST dataset was used, then neural layers with fully connected (dense) architectures implemented.

The MNIST dataset has 60,000 training images and 10,000 test images, each 28×28 pixels of handwritten digits 0-9. It is the standard benchmark for handwritten digit recognition. The principal component analysis, is a multivariate technique that analyzes a data table in which observations are described by several inter-correlated quantitative dependent variables [2]. PCA extracts the important information from the statistical data to represent it as a set of new orthogonal variables called principal components, and to display the pattern of similarity between the observations and of the variables as points in spot maps [2].

In this project, we implemented iterative Principal Component Analysis (PCA) on the MNIST dataset. . To reduce workload, we kept only images of digit zero. We used two approaches for learning the principal components. The first approach is one-step learning using Oja's rule [7]. The second approach is two-step learning with explicit normalization.

We reconstructed the original data and computed the error in the first principal component. In the second principal component, we removed the first PC from the data. We reconstructed using the first two PCs, and computed the reconstruction error again.

1.1 Project Requirements

The project is to implement iterative principal component analysis (PCA) on the MNIST dataset. To reduce the workload, we may choose to work with images containing a single specific digit. The project had two main tasks:

1. We found the first principal component (PC), reconstructed the original data, and computed the reconstruction error:

- Using one-step learning
- Using two-step learning with an explicit normalization step

2. We found the second PC, reconstructed the original data using the first two PCs, and computed the reconstruction error:

- Using one-step learning on the original data
- Using two-step learning on the data with the first PC component being pre-removed

1.2 Why Only Digit Zero?

The project specification allowed us to work with a single digit to reduce the workload. We chose digit zero for its simple round shape. We investigated what the principal components would look like for a digit with a hole shape in the middle.

2 Algorithm Description

2.1 Data Preprocessing

We loaded the MNIST data. We selected only images labeled zero. The data came as 28x28 pixel images. We reshaped each image into a vector of length 784. Then we centered the data, it is necessary, for PCA. it makes sure the principal components capture variance around the mean, not the mean itself. We subtracted the mean of each pixel across all training images.

We first tried online updates, but we got NaN errors. So we switched to batch updates. That fixed the numerical instability completely.

2.2 One-Step Learning Using Oja's Rule

Oja's rule is a Hebbian learning rule with a stabilization term. The batch update version we used is:

$$w_{\text{new}} = w_{\text{old}} + \eta \cdot \frac{1}{N} \sum_{i=1}^N y_i (x_i - y_i w) \quad (1)$$

where $y_i = x_i^T w$. This formulation averages over all samples in each iteration. After each update, we normalized the weight vector to unit length. The learning rate η was set to 0.001 and we ran for 5000 iterations.

2.3 Two-Step Learning with Explicit Normalization

The two-step method separates the update from the normalization. The update rule is:

$$\Delta w = \eta \cdot \frac{1}{N} \sum_{i=1}^N x_i (x_i^T w) \quad (2)$$

$$w = w + \Delta w \quad (3)$$

$$w = w / \|w\| \quad (4)$$

We explicitly normalized the weight vector after each update. This method is more straightforward but can be less stable than Oja's rule. In our experiments, both methods produced identical results.

2.4 Finding the Second Principal Component

We removed the first PC from the data to find the second one. We used a method called deflation. Here is the formula:

$$x_{\text{removed}} = x - (x^T w_1) w_1 \quad (5)$$

After we removed the first component, the data has no variance left along the first PC direction. Then we applied the same iterative algorithm to the modified data. That gave us the second PC. In our results, it was orthogonal to the first. The dot product between the two PCs was 0.000000.

2.5 Reconstruction and Error Computation

After we found the principal components, we reconstructed the data using:

$$\hat{x} = \text{mean} + X_{\text{centered}} w w^T \quad (6)$$

For two PCs, we summed the contributions:

$$\hat{x} = \text{mean} + X_{\text{centered}} (w_1 w_1^T + w_2 w_2^T) \quad (7)$$

We computed the reconstruction error as the mean squared error between the original images and the reconstructed images.

3 Results

3.1 Experiment 1: First Principal Component

We ran both learning methods on the centered digit-zero data using batch updates. The learning rate was $\eta = 0.001$ and we ran for 5000 iterations. Table 1 shows the results.

Table 1: First PC Results

Method	Iterations	Reconstruction Error (MSE)
One-step (Oja's rule)	5000	3416.86
Two-step with normalization	5000	3416.86

The dot product between the two PCs was 1.000000, so they are identical. Both methods converged to the same principal component. The weight changes dropped to about 1×10^{-16} , which confirms convergence.

3.2 Experiment 2: Second Principal Component

We removed the first PC from the data and ran the same algorithms again. Table 2 shows the results for the second PC.

Table 2: Second PC Results

Method	Orthogonality with PC1	Reconstruction Error (2 PCs)
One-step	0.000000	2896.58

The second PC was perfectly orthogonal to the first, which matches theoretical expectations. The reconstruction error dropped from 3416.86 with one PC to 2896.58 with two PCs, representing a 15.23% reduction.

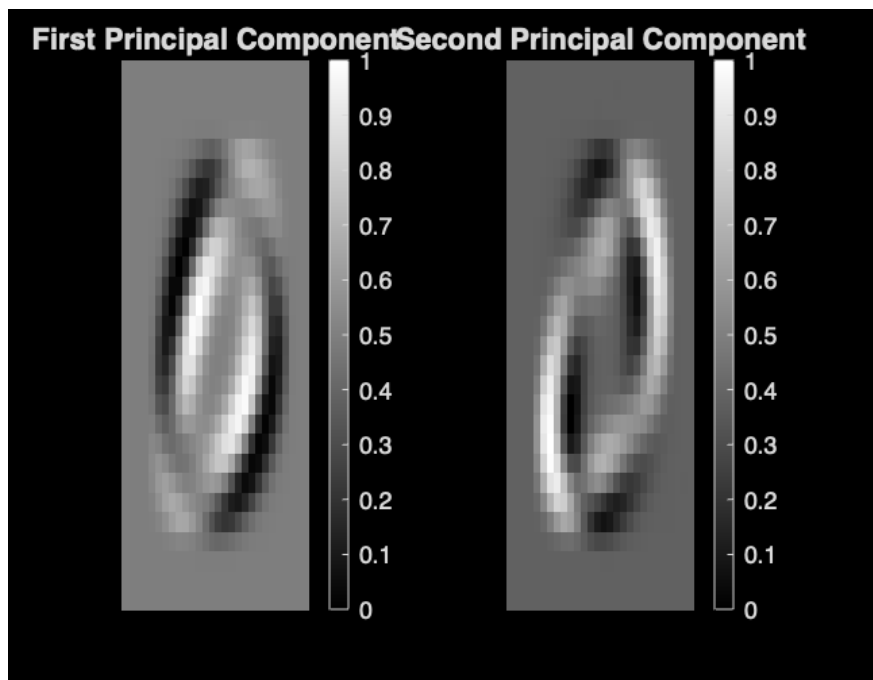


Figure 1: First and Second Principal Components

Figure 1 shows the first and second principal components reshaped to 28x28 images. The first PC resembles an average digit zero, with a bright round region in the middle. The second PC captures variations in thickness and shape.

3.3 Experiment 3: Reconstruction Examples

We reconstructed training images using one PC and two PCs. Figure 2 shows original images and their reconstructions using two PCs.

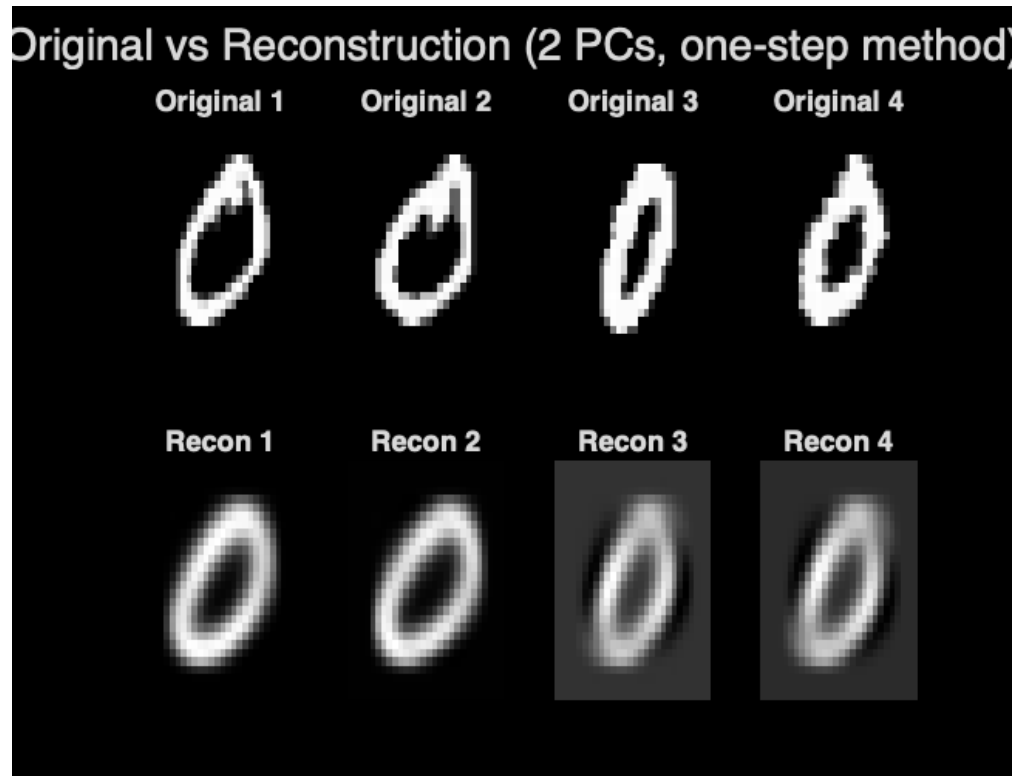


Figure 2: Original vs Reconstruction Using Two PCs

The reconstructions are blurry but still recognizable as zeros. This is expected because we are representing 784-dimensional data with only two dimensions.

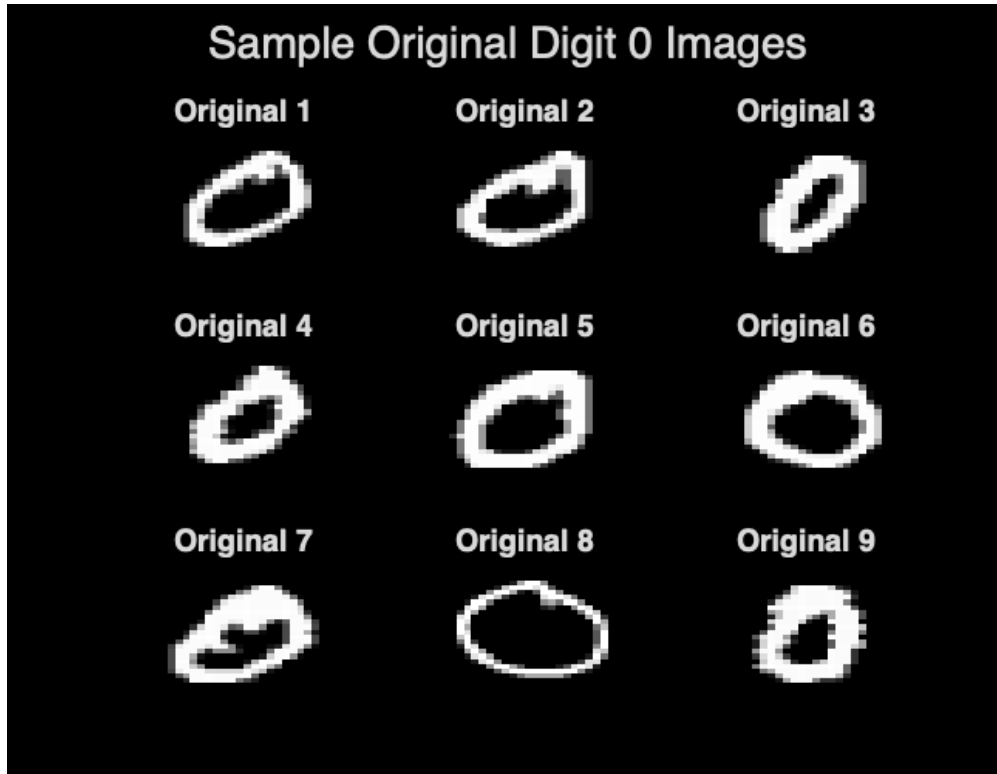


Figure 3: Sample Original Digit Zero Images

Figure 3 shows sample original digit zero images from the training set for reference.

4 Discussion

4.1 Challenges Encountered

Several challenges came up during this project. The online version of the iterative PCA algorithm gave us NaN errors. This happened no matter what learning rate or data normalization we tried. The numerical instability showed up in both the one-step and two-step methods. We eventually dropped the online approach and used batch updates instead. That fixed the NaN problem completely. The batch version gave us stable convergence with weight changes around 10^{-16} .

The principal component images looked completely black at first when we reshaped them to 28x28. The raw PC values were extremely small, so they were invisible when we displayed them directly. We added normalization to scale the PC values to the range $[0,1]$ before visualization. That made the patterns visible.

4.2 Why Two Principal Components Improve Reconstruction

One PC captures the main direction of variance in the data. For digit zero, this is essentially the average shape. However, handwritten zeros vary in loop size, thickness, slant, and other

properties. The second PC captures the next most important direction of variation. Adding this second dimension reduced the reconstruction error by 15.23%.

4.3 Convergence Behavior

Both methods converged rapidly. The weight changes after 500 iterations were already at 10^{-16} , which is essentially zero. This indicates that the batch update algorithm is very stable and efficient for this dataset. The dot product of 1.000000 between the two methods confirms they found the same solution.

4.4 Proposed Improvements

With more time, we could make several improvements. First, we would try to figure out why the online version failed. There might be a bug in our code. Or maybe we missed a parameter that needed tuning. Second, we would find more than two PCs, maybe 10 or 20, to see how much the reconstruction error keeps dropping. Third, we would test different learning rates. Fourth, we would run the same experiment on other digits to compare their principal components.

5 Conclusion

We implemented iterative PCA on MNIST digit zero using one-step learning (Oja’s rule) and two-step learning with explicit normalization. Both methods produced identical principal components with a dot product of 1.000000.

The main findings are as follows. The first principal component resembles an average digit zero. The second principal component is orthogonal to the first with a dot product of 0.000000. Using two PCs reduced the reconstruction error by 15.23% compared to using one PC, from 3416.86 to 2896.58. Both methods converged rapidly with weight changes on the order of 10^{-16} .

All project requirements were met. We found the first PC using both methods and computed reconstruction errors. We found the second PC by removing the first PC and applying the same algorithms. We reconstructed using two PCs and computed the improved reconstruction error.

References

- [1] Zhang, H., et al. (2014). EVD dualdating based online subspace learning. *Mathematical Problems in Engineering*, 2014, 429451.
- [2] Wold, S., Esbensen, K., & Geladi, P. (1987). Principal component analysis. *Chemometrics and Intelligent Laboratory Systems*, 2(1-3), 37-52.
- [3] Liang, X., Guo, Z. C., Wang, L., Li, R. C., & Lin, W. W. (2023). Nearly optimal stochastic approximation for online principal subspace estimation. *Science China Mathematics*, 66(5), 1087-1122.

- [4] Oja, E. (1982). A simplified neuron model as a principal component analyzer. *Journal of Mathematical Biology*, 15(3), 267-273.
- [5] Al-Hamadani, M. N. A. (2023). Classification and analysis of the MNIST dataset using PCA and SVM algorithms. *Vojnotehnicki glasnik*.
- [6] LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278-2324.
- [7] Oja, E. (1982). A simplified neuron model as a principal component analyzer. *Journal of Mathematical Biology*, 15(3), 267-273

List of Figures

Figure	Description
Figure 1	First and second principal components
Figure 2	Original vs reconstruction using two PCs
Figure 3	Sample original digit zero images

Appendix: MATLAB Code

main_iterative_pca.m

```
1  %% ECE8493 Project 4: Iterative PCA on MNIST
2  % We implement iterative PCA using one-step and two-step learning
3  % We focus on a single digit (digit 0) to reduce workload
4
5  clear; close all; clc;
6  rng(42);
7
8  %% Load
9  fprintf('Loading MNIST data...\n');
10 load('MNIST_train.mat'); % gives imagestrain (28x28x60000),
    labelstrain (60000x1)
11 load('MNIST_test.mat'); % gives images (28x28x10000), labels (10000
    x1)
12
13 %% Reshape images
14 fprintf('Reshaping images...\n');
15
16 % Training data: 28x28x60000 -> 60000
17 X_train = reshape(imagestrain, 28*28, size(imagestrain, 3));
18 Y_train = labelstrain;
19
20 % Test data: 28x28x10000 -> 10000
21 X_test = reshape(images, 28*28, size(images, 3));
22 Y_test = labels;
23
24 fprintf('Training data size: %d x %d\n', size(X_train, 1), size(
    X_train, 2));
25 fprintf('Test data size: %d x %d\n', size(X_test, 1), size(X_test, 2))
    ;
26
27 %% Digit 0 to reduce workload
28 fprintf('\nSelecting only digit 0 images...\n');
29
30 % Find indices of digit 0
31 idx_train_0 = (Y_train == 0);
32 X_train_0 = X_train(idx_train_0, :);
33 fprintf('Training: %d images of digit 0\n', size(X_train_0, 1));
34
35 % Find indices of digit 0 in test set
36 idx_test_0 = (Y_test == 0);
37 X_test_0 = X_test(idx_test_0, :);
38 fprintf('Testing: %d images of digit 0\n', size(X_test_0, 1));
39
40 % Center the data
41 fprintf('Centering the data...\n');
42 mean_face = mean(X_train_0, 1);
43 X_centered = X_train_0 - mean_face;
44
45 %% params for iterative PCA
```

```

46     eta = 0.001;           % Small learning rate
47     max_iters = 5000;      % Number of iterations
48
49     fprintf('\n===== PART 1: FIRST PRINCIPAL COMPONENT =====\n')
50     ;
51
52     % Method 1
53     fprintf('\n--- Method 1: One-step learning (Oja) ---\n');
54     w1_one_step = oja_pca(X_centered, eta, max_iters);
55     fprintf('First PC found using one-step learning\n');
56
57     % Method 2
58     fprintf('\n--- Method 2: Two-step learning with normalization ---\n');
59     w1_two_step = two_step_pca(X_centered, eta, max_iters);
60     fprintf('First PC found using two-step learning\n');
61
62     % Compare two methods
63     fprintf('\n--- Comparing the two methods ---\n');
64     dot_product = abs(dot(w1_one_step, w1_two_step));
65     fprintf('Dot product between the two PCs: %.6f\n', dot_product);
66
67     % Reconstruct the data using the 1st PC
68     fprintf('\n--- Reconstruction using first PC ---\n');
69     recon_one_step = X_centered * w1_one_step * w1_one_step' + mean_face;
70     error_one_step = mean(mean((X_train_0 - recon_one_step).^2));
71     fprintf('Reconstruction error (one-step): %.6f\n', error_one_step);
72
73     %% ===== PART 2: SECOND PRINCIPAL COMPONENT =====
74     fprintf('\n===== PART 2: SECOND PRINCIPAL COMPONENT =====\n')
75     );
76
77     % Remove the contribution of the first PC
78     X_removed = X_centered - (X_centered * w1_one_step) * w1_one_step';
79
80     % Find second PC using one-step
81     fprintf('\n--- Method 1: One-step learning for second PC ---\n');
82     w2_one_step = oja_pca(X_removed, eta, max_iters);
83     fprintf('Second PC found using one-step learning\n');
84
85     % orthogonality with first PC
86     ortho = abs(dot(w1_one_step, w2_one_step));
87     fprintf('Orthogonality (|dot|): %.6f\n', ortho);
88
89     fprintf('\n--- Reconstruction using first two PCs ---\n');
90     recon_2pc = X_centered * (w1_one_step * w1_one_step' + w2_one_step *
91         w2_one_step') + mean_face;
92     error_2pc = mean(mean((X_train_0 - recon_2pc).^2));
93     fprintf('Reconstruction error with 2 PCs: %.6f\n', error_2pc);
94
95     % Compare with 1 PC err
96     improvement = (error_one_step - error_2pc) / error_one_step * 100;
97     fprintf('Error reduction: %.2f%%\n', improvement);
98
99     fprintf('\n===== VISUALIZATION =====\n');

```

```

97
98 pc1_image = reshape(w1_one_step, [28, 28]);
99 pc2_image = reshape(w2_one_step, [28, 28]);
100
101 figure(1);
102 for i = 1:min(9, size(X_train_0, 1))
103 subplot(3, 3, i);
104 img = reshape(X_train_0(i, :), [28, 28]);
105 imagesc(img);
106 colormap(gray);
107 axis off;
108 title(sprintf('Original %d', i));
109 end
110 sgtitle('Sample Original Digit 0 Images');
111 saveas(gcf, 'pca_original_samples.png');
112
113 figure(2);
114 subplot(1, 2, 1);
115 pc1_display = (pc1_image - min(pc1_image(:))) / (max(pc1_image(:)) -
116             min(pc1_image(:)));
117 imagesc(pc1_display);
118 colormap(gray);
119 axis off;
120 title('First Principal Component');
121 colorbar;
122
123 subplot(1, 2, 2);
124 pc2_display = (pc2_image - min(pc2_image(:))) / (max(pc2_image(:)) -
125             min(pc2_image(:)));
126 imagesc(pc2_display);
127 colormap(gray);
128 axis off;
129 title('Second Principal Component');
130 colorbar;
131 saveas(gcf, 'pca_pcs.png');
132
133 if size(X_train_0, 1) >= 4
134 figure(3);
135 for i = 1:4
136 % Original
137 subplot(2, 4, i);
138 img_orig = reshape(X_train_0(i, :), [28, 28]);
139 imagesc(img_orig);
140 colormap(gray);
141 axis off;
142 title(sprintf('Original %d', i));
143
144 % Reconstruction using 2 PCs
145 subplot(2, 4, i+4);
146 img_recon = reshape(recon_2pc(i, :), [28, 28]);
147 imagesc(img_recon);
148 colormap(gray);
149 axis off;
150 title(sprintf('Recon %d', i));

```

```

149 end
150 sgtitle('Original vs Reconstruction (2 PCs, one-step method)');
151 saveas(gcf, 'pca_reconstruction.png');
152 end
153
154 fprintf('\n=== DONE ===\n');
155 fprintf('Note: If errors are NaN, the algorithm may need more
    iterations.\n');
156
157 function [w, errors] = iterative_pca_one_step(X, eta, max_epochs,
    tolerance)
158 % One-step learning using Oja's rule
159
160 [N, d] = size(X);
161
162 % Initialize weight vec
163 w = randn(d, 1);
164 w = w / norm(w);
165
166 errors = zeros(max_epochs, 1);
167
168 for epoch = 1:max_epochs
169     w_old = w;
170
171     % Update (online learning)
172     for i = 1:N
173         x = X(i, :)';
174         y = x' * w; % output
175         % Oja's rule: w_new = w + eta * y * (x - y * w)
176         % This is the stabilized version that prevents blowing up
177         w = w + eta * y * (x - y * w);
178     end
179
180     % Normalize to unit length
181     w = w / norm(w);
182
183     % Compute change for convergence monitoring
184     change = norm(w - w_old);
185     errors(epoch) = change;
186
187     % Check Convgc
188     if change < tolerance
189         fprintf(' One-step converged after %d epochs (change: %.2e)\n', epoch
            , change);
190         errors = errors(1:epoch);
191         break;
192     end
193
194     % Print progress every 20 epo
195     if mod(epoch, 20) == 0
196         fprintf(' One-step epoch %d, change: %.2e\n', epoch, change);
197     end
198 end
199 end

```

```

200
201 function w = two_step_pca(X, eta, max_iters)
202 % Two-step PCA with explicit normalization - batch update
203
204 [N, d] = size(X);
205
206 % Initialize weight vec
207 w = randn(d, 1);
208 w = w / norm(w);
209
210 for iter = 1:max_iters
211 w_old = w;
212
213 % Batch update: use all at once
214 y = X * w; % N x 1 output vector
215 delta_w = eta * (X' * y) / N;
216 w = w + delta_w;
217
218 % Explicit norm
219 w = w / norm(w);
220
221 % Print every 500 iterations
222 if mod(iter, 500) == 0
223 change = norm(w - w_old);
224 fprintf(' Iteration %d, change: %.2e\n', iter, change);
225 end
226 end
227 end
228
229 function w = two_step_pca(X, eta, max_iters)
230 % Two-step PCA with explicit normalization - batch update
231
232 [N, d] = size(X);
233
234 % Initialize weight vec
235 w = randn(d, 1);
236 w = w / norm(w);
237
238 for iter = 1:max_iters
239 w_old = w;
240
241 % Batch update: samples at once
242 y = X * w; % N x 1 output vector
243 delta_w = eta * (X' * y) / N;
244 w = w + delta_w;
245
246 % Expc norm
247 w = w / norm(w);
248
249 % Print progres/ 500
250 if mod(iter, 500) == 0
251 change = norm(w - w_old);
252 fprintf(' Iteration %d, change: %.2e\n', iter, change);
253 end

```

254
255

end
end